

Nome: _____ Número: _____

Perg. 1 => Resposta no enunciado Perg. 2 => Folha de PROVA Perg. 3 => Folha de CONTINUAÇÃO

1. [15%] O código abaixo utiliza a classe Log que apenas imprime mensagens quando é construída e destruída, para monitorizar a gestão de memória.

a) Complete os espaços em branco (_____) para que o código compile e funcione corretamente, respeitando a lógica dos smart pointers.

```
class Log {  
    int id;  
public:  
    Log(int i) : id(i) { std::cout << "Criado:" << id << "\n"; }  
    ~Log() { std::cout << "Destruído: " << id << "\n"; }  
    void metodo() { ... }  
};
```

```
int main() {  
    // Deve usar a função auxiliar moderna recomendada (não use 'new').  
    std::unique_ptr<Log> sp1 = std::_____(10);  
  
    std::vector<std::unique_ptr<Log>> lista;  
  
    // Adicionar 'sp1' ao vetor 'lista'.  
    lista.push_back(std::_____(sp1));  
  
    // Deve usar a função auxiliar moderna recomendada (não use 'new').  
    std::shared_ptr<Log> sp2 = std::_____(20);  
  
    // Criação de um ponteiro que não impede a destruição do objeto.  
    std::_____ sp3 = sp2;  
  
    // Queremos chamar o método 'metodo()' do objeto apontado por sp3.  
    if (auto tempPtr = sp3._____) {  
        tempPtr->metodo();  
    }  
  
    return 0;  
}
```

b) Considere o seguinte excerto de código independente. **Indique exatamente o output gerado pela execução da função teste().**

```
void analisar(std::shared_ptr<Log> p) {  
    if (p) {  
        std::cout << "Ref: " << p.use_count() << "\n";  
    }  
}  
  
void teste() {  
    std::shared_ptr<Log> pA = std::make_shared<Log>(1);  
    {  
        std::shared_ptr<Log> pB = pA;  
        analisar(pB);  
        std::shared_ptr<Log> pC = std::make_shared<Log>(2);  
        std::cout << "fim bloco\n";  
    }  
    std::cout << "fim funcao\n";  
}
```

Resposta neste retângulo

2. [75%] Pretende-se desenvolver algumas classes em C++ para simular **cozinhados** de culinária, os quais envolvem **Ingredientes** de vários tipos na sua confeção. O exercício está estruturado em alíneas. Cada alínea pode ser feita de forma independente, mas aconselha-se muito a ler o enunciado de todas antes de começar a responder.

- Não precisa separar o seu código em .h e .cpp
- Não precisa de especificar #includes nem namespaces de coisas standard

a) *Ingrediente*: Um ingrediente é um conceito genérico que diz respeito a algo que pertencerá mais tarde a um cozinhado. Os ingredientes exibem um comportamento que depende do tipo concreto de ingrediente que é.

Todos os ingredientes têm uma quantidade em gramas (`int`), um estado (`string`) que pode ser "sólido", "líquido" ou "gasoso", e um nível de acidez (`pH` – `double`) que pode variar entre 0 e 14.

A criação do ingrediente presume a indicação dos seus dados. Todas as características dos ingredientes podem ser obtidas e, à partida, modificadas. Deve ser possível obter uma representação textual (`string`) do ingrediente contendo informações que podem depender do ingrediente em questão.

A cada minuto que passa num cozinhado, acontece algo ao ingrediente ou ao cozinhado, dependendo do ingrediente. O ingrediente pode alterar as suas próprias propriedades ou as do cozinhado, o que significa que o método que concretiza esta funcionalidade recebe por parâmetro o cozinhado.

Dos muitos ingredientes que existem, este enunciado pede apenas o ingrediente **Água**. Este ingrediente tem as características comuns a todos os ingredientes, com as seguintes diferenças:

- O nível de acidez (`pH`) da água é sempre 7.
- Se tiver 5 gramas ou menos, não faz nada. Caso contrário, a cada minuto que passa à temperatura superior ou igual a 50°C, perde 10% (arredondado) do seu peso, e se atingir 100°C, muda o primeiro ingrediente sólido (se existir algum) do cozinhado para líquido e fica ele próprio (a água) com 1 grama.
- A descrição dela inclui a frase "água da fonte".

Implemente o ingrediente **Água**, incluindo outras classes que considere necessárias.

b) *Cozinhado*: Um cozinhado tem uma temperatura e um conjunto de tamanho variável, sem limite à partida, de ingredientes de qualquer tipo. Pode haver ingredientes repetidos. Inicialmente, a temperatura do cozinhado é 20 e não tem qualquer ingrediente. O cozinhado permite as seguintes operações:

- Adicionar um ingrediente: `void adiciona(...)`. Esta função recebe um ingrediente já feito – não são os dados para o fazer. O cozinhado deverá guardar uma cópia do Ingrediente e não o objecto original. Esta função é a mesma para qualquer ingrediente.
- Simular mais um minuto de cozedura: `void cozinha(int temp)`. Esta função recebe a temperatura a que esse minuto de cozedura decorre, podendo chamadas sucessivas indicar temperaturas completamente diferentes. Considera-se que a temperatura do cozinhado é a que foi indicada na última chamada a este método.

A simulação do minuto de cozedura deve garantir que o efeito dos ingredientes relativos a mais um minuto a uma determinada temperatura é despoletado – se não tiver feito a alínea anterior, assuma que tem o código resultante dela e invoque os métodos que considerar necessários.

- Remover ingredientes ácidos: `void removeAcidos()`. Este método remove todos os ingredientes cujo nível de acidez seja inferior a 7 ($pH < 7$).
- Obter a lista com a descrição de todos os ingredientes (mesmo que tenham 0 gramas): `string obtenDescricao()`.
- Obter a temperatura atual com um método chamado `obtemTemp`.
- Obter o número de ingredientes com um método cujo nome é `obtemTotal`.
- Obter acesso de consulta e alteração a um ingrediente dentro de um cozinhado dado o índice desse ingrediente. O primeiro ingrediente tem o índice 0. O método chama-se `ingredienteEm`, recebe o índice.

As funções que não alteram o cozinhado devem poder ser chamadas sobre objetos constantes.

Os cozinhados são proprietários exclusivos dos seus ingredientes, e objetos *Cozinhado* comportam-se como objetos normais de C++, podendo ser copiados, atribuídos e passados por valor, mantendo sempre ingredientes independentes.

Escreva a classe *Cozinhado*.

- c) Pretende-se que seja possível operar sobre objetos Cozinhado usando expressões tais como as que estão nos exemplos abaixo. Se não tiver feito a alínea anterior, assuma que os métodos nela referidos existem.

No exemplo abaixo, assuma que, para além das classes referentes às alíneas anteriores, estão definidas as classes Massa e Farinha.

```
Agua a(50, "líquido");  
Massa m(100, "sólido", 6, 20); // 100g de massa algo ácida, com 20 cm  
Farinha f(50, "sólida", 8);    // 50g de farinha ligeiramente alcalina  
Cozinhado c;
```

```
// Adicionar ingredientes a um cozinhado:
```

```
c << a << m << f;    // não adiciona ingredientes uns aos outros:  
                        // adiciona ingredientes um por um ao cozinhado
```

```
c += 30; // Cozinhado c passa mais um minuto a 30 graus:
```

```
// Saber se o cozinhado tem pelo menos um ingrediente
```

```
// (mesmo que tenha 0 gramas):
```

```
if (c) { std::cout << "o cozinhado não está vazio"; }
```

Escreva o código necessário para que estas expressões sejam possíveis, **indicando**, caso a caso, a que classe pertence ou se não pertence a nenhuma.

3. [10%] Identifique, explicando, os erros no código abaixo, quer de sintaxe (se houver), quer de lógica de orientação a objetos. Não precisa de transcrever o código todo e pode usar o número de linha na identificação.

```
01 struct Resumo {
02     char letra;
03     int paginas;
04     Resumo(char c='A', int p=0) : letra(c), paginas(p) {}
05     virtual ~Resumo() { }
06 };
07
08 class Livro {
09     std::string titulo;
10     Resumo resumo;
11
12     Livro(std::string titulo, char c, int p) : resumo(c,p){titulo=titulo;}
13     Livro() {}
14     virtual ~Livro() { }
15
16     std::string getTitulo() const { return titulo; }
17     int& getPaginas() const { return resumo.paginas; }
18
19     void setTitulo(const std::string t) const { titulo = t; }
20
21 };
22
23
24 int main() {
25     Livro l1("O Diário da Ema", 'A', 100);
26     Livro l2;
27
28     l1.setTitulo("O Diário da Sofia");
29     l1.getPaginas() = 150;
30     l2 = l1;
31
32     Resumo resumo_1('B',50);
33     resumo_1.paginas = 60;
34
35     return 0;
36 }
```